

INTERVIEW PREPARATION SERIES

Sorting Algorithms Interview Guide

Core theory, patterns, and worked examples for every major sorting algorithm asked in coding interviews

- ▶ Fundamentals & complexity analysis
- ▶ 9 core sorting algorithms
- ▶ Worked examples with step-by-step trace
- ▶ Curated interview question bank by pattern
- ▶ Quick-reference complexity cheat sheet

1. Sorting Fundamentals

What is Sorting?

Sorting is the process of arranging elements of a collection into a defined order, usually ascending or descending. It's rarely the end goal in an interview — it's almost always a stepping stone that unlocks a faster algorithm elsewhere (binary search, two pointers, greedy scheduling). Interviewers test sorting knowledge in two ways: implementing an algorithm from scratch (to check you understand the mechanics), and recognizing when sorting first simplifies a harder problem.

WHY INTERVIEWERS LOVE SORTING

Sorting algorithms are a compact way to test multiple skills at once: recursion and divide-and-conquer (merge sort, quick sort), in-place array manipulation (heap sort, quicksort partitioning), understanding of best/average/worst case analysis, and the idea that you can trade comparison-based limits ($O(n \log n)$) for linear time when you know something extra about your data (counting sort, radix sort).

Comparison-Based vs Non-Comparison-Based

Comparison sorts (bubble, insertion, selection, merge, quick, heap) only ever ask "is $A < B$?" and are provably bounded below by $O(n \log n)$ in the worst case. Non-comparison sorts (counting, radix, bucket) exploit extra knowledge about the values themselves — such as a bounded integer range — to sort in linear time, at the cost of extra space or restrictions on the input.

Key Vocabulary

Term	Meaning
Stable	Equal elements keep their original relative order after sorting
In-place	Uses $O(1)$ or $O(\log n)$ extra space, sorts within the original array
Adaptive	Runs faster when the input is already partially sorted
Comparison-based	Only compares pairs of elements; lower bound is $O(n \log n)$

How to Approach a Sorting Question

1. **Clarify what's being asked:** implement a specific algorithm from scratch, or use sorting as a subroutine to solve a bigger problem?
2. **Ask about the data:** size of n , are values bounded integers (opens the door to counting/radix sort), is stability required, is extra memory allowed?
3. **If implementing from scratch:** state the invariant of your algorithm before coding (e.g. "after pass i , the i largest elements are in their final position").
4. **If using sorting as a tool:** sort first, then ask what structure that exposes — adjacent duplicates, two-pointer opportunities, or a greedy ordering.
5. **Trace on a small, partly-adversarial example** (some duplicates, a couple already in place) to catch bugs in swap/partition logic.
6. **State time, space, and stability** without being asked — interviewers use this to confirm real understanding, not memorization.

2. Core Sorting Algorithms

Below are the algorithms that cover the overwhelming majority of sorting interview questions. For each: the theory, when to use it, a worked example with a step-by-step trace, and complexity.

2.1 Bubble Sort

Theory: Repeatedly step through the array, swapping adjacent elements if they're in the wrong order. Each full pass "bubbles" the largest remaining element to its correct position at the end. Stop early if a pass makes no swaps — the array is already sorted.

WHEN TO USE

Almost never in production, but a common warm-up question to check basic looping/swapping logic and awareness of the early-exit optimization.

WORKED EXAMPLE: BUBBLE SORT

```
# arr = [5, 1, 4, 2, 8]
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break          # already sorted, stop early
    return arr
```

Trace: Pass 1: [5,1,4,2,8]→swap(5,1)→[1,5,4,2,8]→swap(5,4)→[1,4,5,2,8]→swap(5,2)→[1,4,2,5,8]→(5,8) no swap. Pass 2: [1,4,2,5,8]→swap(4,2)→[1,2,4,5,8]. Pass 3: no swaps → break early. Sorted: [1,2,4,5,8].

COMPLEXITY

Time: $O(n^2)$ worst/average, $O(n)$ best (already sorted, with early exit) | Space: $O(1)$ | Stable: Yes | In-place: Yes

2.2 Selection Sort

Theory: Divide the array into a sorted prefix and an unsorted suffix. On each pass, find the minimum element in the unsorted suffix and swap it into the front of that suffix, growing the sorted prefix by one.

WHEN TO USE

When the cost of swaps matters more than comparisons (selection sort does at most n swaps total, unlike bubble/insertion sort) — e.g. sorting data where writes are expensive (flash memory).

WORKED EXAMPLE: SELECTION SORT

```
# arr = [29, 10, 14, 37, 13]
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

Trace: i=0: min in [29,10,14,37,13] is 10(idx1) → swap → [10,29,14,37,13]. i=1: min in [29,14,37,13] is 13(idx4) → swap → [10,13,14,37,29]. i=2: min in [14,37,29] is 14, already in place. i=3: min in [37,29] is 29(idx4) → swap → [10,13,14,29,37]. Sorted.

COMPLEXITY

Time: $O(n^2)$ in all cases (always scans the full remaining suffix) | Space: $O(1)$ | Stable: No (standard swap-based version) | In-place: Yes

2.3 Insertion Sort

Theory: Build the sorted array one element at a time. Take the next unsorted element and shift it leftward through the sorted prefix until it lands in its correct position — exactly like sorting playing cards in your hand.

WHEN TO USE

Small arrays, nearly-sorted data (it's highly adaptive — $O(n)$ when input is already almost sorted), or as the base case inside hybrid algorithms like Timsort and introsort.

WORKED EXAMPLE: INSERTION SORT

```
# arr = [12, 11, 13, 5, 6]
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]    # shift right
            j -= 1
        arr[j + 1] = key
    return arr
```

Trace: $i=1$, $key=11$: $12 > 11$ shift $\rightarrow [12, 12, 13, 5, 6] \rightarrow$ place 11 $\rightarrow [11, 12, 13, 5, 6]$. $i=2$, $key=13$: $12 < 13$, no shift $\rightarrow [11, 12, 13, 5, 6]$. $i=3$, $key=5$: shift 13, 12, 11 right $\rightarrow$ place 5 at front $\rightarrow [5, 11, 12, 13, 6]$. $i=4$, $key=6$: shift 13, 12, 11 right $\rightarrow$ place 6 after 5 $\rightarrow [5, 6, 11, 12, 13]$. Sorted.

COMPLEXITY

Time: $O(n^2)$ worst/average, $O(n)$ best (nearly sorted input) | Space: $O(1)$ | Stable: Yes | In-place: Yes

2.4 Merge Sort

Theory: A classic divide-and-conquer algorithm. Split the array in half recursively until each piece has one element (trivially sorted), then merge pairs of sorted halves back together in linear time. The merge step is the crux: walk two sorted lists with two pointers, always taking the smaller front element.

WHEN TO USE

When you need guaranteed $O(n \log n)$ worst case and stability (e.g. sorting linked lists, external sorting of data too big for memory, counting inversions in an array).

WORKED EXAMPLE: MERGE SORT

```
# arr = [38, 27, 43, 3, 9, 82, 10]
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

Trace: Split $[38, 27, 43, 3, 9, 82, 10] \rightarrow [38, 27, 43]$ and $[3, 9, 82, 10] \rightarrow$ recursively split down to single elements $\rightarrow$ merge back up: $[27, 38]$, $[3, 43]$, $[9, 82, 10] \rightarrow \dots$ eventually merging $[3, 27, 38, 43]$ with $[3, 9, 10, 82]$ style pairs to yield $[3, 9, 10, 27, 38, 43, 82]$.

COMPLEXITY

Time: $O(n \log n)$ in all cases | Space: $O(n)$ auxiliary | Stable: Yes | In-place: No

2.5 Quick Sort

Theory: Also divide-and-conquer, but the work happens before the recursion instead of after. Pick a pivot, partition the array so everything smaller than the pivot ends up on its left and everything larger on its right, then recursively sort each side. Unlike merge sort, this can be done in-place.

WHEN TO USE

General-purpose sorting when average-case speed and in-place, low-memory operation matter more than worst-case guarantees. Also the basis for quickselect (finding the k th smallest/largest element in $O(n)$ average time).

WORKED EXAMPLE: QUICK SORT (LOMUTO PARTITION)

```
# arr = [10, 7, 8, 9, 1, 5]
```

```
def quick_sort(arr, lo=0, hi=None):
    if hi is None:
        hi = len(arr) - 1
    if lo < hi:
        p = partition(arr, lo, hi)
        quick_sort(arr, lo, p - 1)
        quick_sort(arr, p + 1, hi)
    return arr

def partition(arr, lo, hi):
    pivot = arr[hi]          # choose last element as pivot
    i = lo - 1
    for j in range(lo, hi):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[hi] = arr[hi], arr[i + 1]
    return i + 1
```

Trace: pivot=5 (last element). Scan j=0..4: 10>5 skip, 7>5 skip, 8>5 skip, 9>5 skip, 1≤5→i=0, swap→[1,7,8,9,10,5]. Place pivot at i+1=1 → swap arr[1] and arr[5] → [1,5,8,9,10,7], pivot index=1. Recurse on [1] (already sorted) and [8,9,10,7] which continues partitioning until fully sorted: [1,5,7,8,9,10].

COMPLEXITY

Time: $O(n \log n)$ average, $O(n^2)$ worst (already-sorted input with a poor pivot choice) | Space: $O(\log n)$ recursion stack | Stable: No | In-place:

Yes

2.6 Heap Sort

Theory: Build a max-heap from the array (so the largest element is always at the root), then repeatedly swap the root with the last unsorted element and "sift down" to restore the heap property, shrinking the heap by one each time. This produces the array sorted in ascending order in-place.

WHEN TO USE

When you need guaranteed $O(n \log n)$ worst case AND in-place $O(1)$ extra space (merge sort needs $O(n)$ space, quicksort's worst case is $O(n^2)$) — heap sort is the only comparison sort offering both guarantees simultaneously. Also directly useful whenever "kth largest" or "top k" comes up.

WORKED EXAMPLE: HEAP SORT

```
# arr = [4, 10, 3, 5, 1]
def heapify(arr, n, i):
    largest = i
    left, right = 2 * i + 1, 2 * i + 2
    if left < n and arr[left] > arr[largest]:
        largest = left
    if right < n and arr[right] > arr[largest]:
        largest = right
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)    # sift down further

def heap_sort(arr):
    n = len(arr)
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)        # build max-heap
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]    # move max to end
        heapify(arr, i, 0)
    return arr
```

Trace: Build max-heap from [4,10,3,5,1] → [10,5,3,4,1]. Swap root with last → [1,5,3,4,10], heapify(0.3) → [5,4,3,1,10]. Swap → [1,4,3,5,10], heapify → [4,1,3,5,10]. Swap → [3,1,4,5,10], heapify → [3,1,4,5,10]. Swap → [1,3,4,5,10]. Sorted: [1,3,4,5,10].

COMPLEXITY

Time: $O(n \log n)$ in all cases | Space: $O(1)$ | Stable: No | In-place: Yes

2.7 Counting Sort

Theory: A non-comparison sort for integers within a known, reasonably small range. Count how many times each value occurs, then use those counts (as a running/prefix sum) to compute exactly where each element belongs in the output — no comparisons needed at all.

WHEN TO USE

Sorting integers with a small range relative to n (e.g. exam scores 0-100, ages), sorting by a single digit/character as a subroutine inside radix sort, or "sort colors / sort an array of 0s, 1s and 2s" style questions.

WORKED EXAMPLE: COUNTING SORT

```
# arr = [4, 2, 2, 8, 3, 3, 1], values in range 0..8
def counting_sort(arr, max_val):
    count = [0] * (max_val + 1)
    for num in arr:
        count[num] += 1
    for i in range(1, len(count)):
        count[i] += count[i - 1] # prefix sum = final position
    output = [0] * len(arr)
    for num in reversed(arr):
        count[num] -= 1
        output[count[num]] = num
    return output
```

Trace: Counts: 1→1, 2→2, 3→2, 4→1, 8→1. Prefix sums: [1,3,5,6,6,6,6,7] (index=value). Placing elements from the back of the input in reverse order fills output as [1,2,2,3,3,4,8], preserving relative order among duplicates.

COMPLEXITY

Time: $O(n + k)$ where k is the value range | Space: $O(n + k)$ | Stable: Yes (with the reversed-scan technique) | In-place: No

2.8 Radix Sort

Theory: Sorts integers digit by digit, from the least significant digit to the most significant, using a stable sort (typically counting sort) as a subroutine at each digit position. Because each pass is stable, sorting by increasingly significant digits progressively refines the order without undoing earlier work.

WHEN TO USE

Sorting large lists of integers or fixed-length strings where the number of digits/characters d is small compared to $\log n$ — e.g. sorting phone numbers, sorting large integers, or sorting strings of equal length.

WORKED EXAMPLE: RADIX SORT (LSD, BASE 10)

```
# arr = [170, 45, 75, 90, 802, 24, 2, 66]
def counting_sort_by_digit(arr, exp):
    output = [0] * len(arr)
    count = [0] * 10
    for num in arr:
        count[(num // exp) % 10] += 1
    for i in range(1, 10):
        count[i] += count[i - 1]
    for num in reversed(arr):
        digit = (num // exp) % 10
        count[digit] -= 1
        output[count[digit]] = num
    return output

def radix_sort(arr):
    max_val = max(arr)
    exp = 1
    while max_val // exp > 0:
        arr = counting_sort_by_digit(arr, exp)
        exp *= 10
    return arr
```

Trace: Sort by 1s digit: [170,90,802,2,24,45,75,66]. Sort that by 10s digit: [802,2,24,45,66,170,75,90]. Sort that by 100s digit: [2,24,45,66,75,90,170,802]. Fully sorted after 3 passes (max value has 3 digits).

COMPLEXITY

Time: $O(d \cdot (n + b))$ where d = number of digits, b = base (10) | Space: $O(n + b)$ | Stable: Yes | In-place: No

2.9 Bucket Sort

Theory: Distribute elements into a fixed number of "buckets" based on value range, sort each bucket individually (typically with insertion sort, since buckets are small), then concatenate the buckets in order. Works best when input values are uniformly distributed over a known range.

WHEN TO USE

Sorting floating-point numbers uniformly distributed in a range like $[0, 1)$, or any data where you can cheaply map a value to an approximate bucket index.

WORKED EXAMPLE: BUCKET SORT

```
# arr = [0.78, 0.17, 0.39, 0.26, 0.72, 0.94, 0.21, 0.12]
def bucket_sort(arr, num_buckets=5):
```

```
buckets = [[] for _ in range(num_buckets)]
for num in arr:
    idx = int(num * num_buckets)      # map value to bucket
    buckets[min(idx, num_buckets - 1)].append(num)
for bucket in buckets:
    bucket.sort()                    # insertion sort in practice
return [num for bucket in buckets for num in bucket]
```

Trace: 5 buckets covering [0,.2),[.2,.4),[.4,.6),[.6,.8),[.8,1). Distribute: bucket0=[0.17,0.12], bucket1=[0.39,0.26,0.21], bucket2=[], bucket3=[0.78,0.72], bucket4=[0.94]. Sort each: [0.12,0.17],[0.21,0.26,0.39],[],[0.72,0.78],[0.94]. Concatenate: fully sorted array.

COMPLEXITY

Time: $O(n + k)$ average (uniform data), $O(n^2)$ worst (all values land in one bucket) | Space: $O(n + k)$ | Stable: Yes (if the per-bucket sort is stable)
| In-place: No

3. Interview Question Bank by Pattern

Use this as a checklist. Practice implementing each algorithm from memory, then solve a few applied questions from each row.

Category	Representative Questions
Implement from Scratch	Implement Bubble/Selection/Insertion Sort Easy , Implement Merge Sort Medium , Implement Quick Sort Medium , Sort an Array (LeetCode 912) Medium
Merge Sort Applications	Count Inversions in an Array Medium , Merge k Sorted Lists Hard , Sort a Linked List Medium , Reverse Pairs Hard
Quick Sort / Partition Applications	Kth Largest Element in an Array (Quickselect) Medium , Sort Colors (Dutch Flag) Medium , Wiggle Sort Medium , Top K Frequent Elements Medium
Heap Sort / Priority Queue	Kth Largest Element in a Stream Easy , Find Median from Data Stream Hard , Top K Frequent Words Medium , Merge k Sorted Arrays Hard
Counting / Radix / Bucket Sort	Sort Colors Medium , Maximum Gap (Radix/Bucket) Hard , Sort an Array by Frequency Easy , H-Index Medium
Sorting as a Subroutine	Merge Intervals Medium , Non-overlapping Intervals Medium , Largest Number (custom comparator) Medium , Meeting Rooms II Medium

4. Interview Tips & Common Pitfalls

Before You Code

Clarify what "sort" means here. Ascending or descending? Stable required? Custom comparator (e.g. sort strings by length then alphabetically)?

Ask about the data's shape. Bounded integers open the door to counting/radix sort for an $O(n)$ solution the interviewer may be fishing for.

State the invariant of your algorithm out loud before coding — e.g. "everything left of the pivot index is smaller" for quicksort's partition step.

Common Pitfalls to Avoid

- **Off-by-one errors in partition logic** — quicksort and heap sort are notorious for subtle index bugs; trace through a 4-5 element example before trusting the code.
- **Forgetting stability requirements** — selection sort, quicksort, and heap sort are not stable by default; if stability is required, use merge sort or a stable variant.
- **Choosing a bad pivot** in quicksort (always picking the first or last element) degrades to $O(n^2)$ on already-sorted or adversarial input — mention randomized or median-of-three pivot selection.
- **Ignoring recursion depth** — merge sort and quicksort both risk stack overflow on very large inputs if implemented naively; mention this trade-off.
- **Applying counting/radix sort to unbounded or negative ranges** without adjusting the algorithm (e.g. offsetting negative values before counting).
- **Reaching for a custom sort when the built-in sort is enough** — in real interviews, unless asked to implement from scratch, using the language's built-in $O(n \log n)$ sort is often the right first move before optimizing further.

During the Interview

- Mention the divide-and-conquer recurrence ($T(n) = 2T(n/2) + O(n)$) if asked to justify merge sort or quicksort's $O(n \log n)$ behavior.
- For quicksort, explain average vs worst case and how pivot choice affects it — this shows depth beyond memorized code.
- After coding, trace through your own example, including duplicates, to confirm correctness live.
- State time, space, and stability without being asked; it signals genuine understanding.

5. Quick Reference: Complexity Cheat Sheet

Algorithm	Best	Average	Worst	Space	Stable
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(n + k)$	Yes
Radix Sort	$O(d(n+b))$	$O(d(n+b))$	$O(d(n+b))$	$O(n + b)$	Yes
Bucket Sort	$O(n + k)$	$O(n + k)$	$O(n^2)$	$O(n + k)$	Yes

Which One Should I Reach For?

Situation	Best Choice
Need guaranteed $O(n \log n)$, stability, and don't mind $O(n)$ space	Merge Sort
Need in-place, good average speed, worst case is acceptable risk	Quick Sort
Need guaranteed $O(n \log n)$ AND $O(1)$ space	Heap Sort
Data is nearly sorted already	Insertion Sort
Integers in a small known range	Counting Sort
Large integers / fixed-length strings	Radix Sort
Uniformly distributed floats in a range	Bucket Sort

FINAL ADVICE

Don't just memorize the code — be able to state, for any algorithm, its invariant, its best/worst case trigger, and why it is or isn't stable. That's what separates a candidate who learned sorting from one who can reason about any new sorting-adjacent problem on the spot.